

¿Aportan información las invariantes geométricas de los grafos de programa por encima del análisis semántico clásico?

MANIFOLD: un estudio falsable sobre OWASP Benchmark 1.2

Proyecto MANIFOLD

Borrador de trabajo v0.4 — no revisado por pares

Abstract

Los pipelines de análisis estático suelen tener el cuello de botella no en *detectar* sinks sino en *decidir cuál verificar*. Planteamos una pregunta deliberadamente falsable: *¿Aportan las invariantes espectrales, topológicas y geométricas de los grafos de programa información sobre el sink vulnerable más allá del análisis semántico clásico (taint)?* MANIFOLD es nuestro instrumento: convierte un artefacto en un grafo tipado (la IR), computa un campo *pre-verificación* $V_{\text{prior}}(x)$ usando solo señales baratas (excluyendo deliberadamente la señal formal, para evitar circularidad), y deja que un ciclo neuro-simbólico actúe sobre las regiones priorizadas. Reportamos un resultado *negativo* riguroso. Sobre OWASP Benchmark 1.2 (2740 casos), el taint crudo ordena mejor el sink vulnerable (MRR 0.894, AUC 0.895) y supera al campo fusionado con $p = 0.000$ (permutación pareada); las señales *espectral* (0.496) y *topológica* (0.500) están en el azar y la *curvatura de Forman–Ricci* está *por debajo* (0.376). En una cola global de todos los candidatos, el campo estructural ayuda solo en el extremo superior ($P@10 = 0.8$ vs. 0.6) pero pierde en precisión media. Las hipótesis H1/H2/H5 quedan por tanto *no soportadas*. Contribuimos: (1) un arnés de oráculo de priorización (MRR, R@1, NDCG@k, AP/P@k/R@k global, consultas-para-90%-recall, test de permutación pareada); (2) un pipeline Java end-to-end reproducible (SAST + verificador Z3 que produce *witnesses verificables por máquina bajo la semántica modelada*); (3) validación externa en la suite Juliet y en commits de fix de CVEs reales, que localiza el cuello de botella real en el taint *interprocedural* y no en más geometría; y (4) una mecanización en Lean 4 de la solidez de la abstracción de taint y de la invariante de autorización.

1 Motivación

Los escáneres tradicionales (SAST/DAST) enumeran patrones sintácticos. Son frágiles: pasan por alto vulnerabilidades novedosas y ahogan al analista en falsos positivos. Faltan dos capacidades:

- (1) Una *representación unificada y cuantitativa* de la “estructura del programa” sobre la cual proyectar y fusionar muchas señales *independientes*.
- (2) Un *ciclo de razonamiento* que convierta “región sospechosa” en “vulnerabilidad probada” — el LLM aporta la intuición, la matemática aporta la prueba.

MANIFOLD proporciona ambas.

2 La idea central

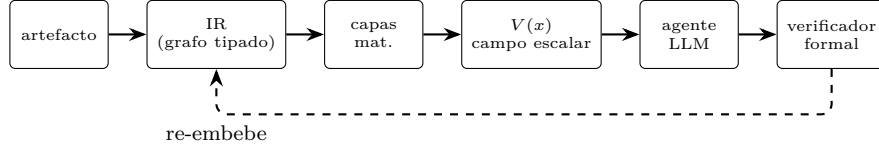


Figure 1: El pipeline: un artefacto se convierte en un grafo tipado, enriquecido con estructura matemática en un campo escalar de vulnerabilidad $V(x)$, que un agente LLM navega; un verificador formal confirma o refuta cada hipótesis y el resultado se re-embebe en el manifold.

Una vulnerabilidad *no* es un patrón; es una *violación de una invariante estructural* (una condición de naturalidad rota, un ciclo persistente, un cuello de botella de curvatura negativa, un flujo de taint que cruza un corte espectral de confianza). Cada capa detecta un *tipo* distinto de violación de invariante, y el campo fusionado $V(x)$ ordena las regiones para que el agente las inspeccione.

3 La Representación Intermedia (IR)

Definición 1 (Grafo de programa). Un *grafo de programa* es una tupla $G = (V, E, \tau_V, \tau_E, \omega)$ donde

- V es un conjunto finito de *nodos* (módulos, funciones, bloques, sentencias, llamadas, variables, fuentes, sumideros);
- $E \subseteq V \times V$ es un conjunto de *aristas* dirigidas;
- $\tau_V : V \rightarrow \mathbb{N}_K$ asigna a cada nodo un *tipo* de $K = \{\text{módulo, función, bloque, sentencia, llamada, asignación, variable, parámetro, fuente, sumidero, gate}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ asigna a cada arista un *tipo* $\mathcal{T} = \{\text{control, dato, llamada, taint, confianza, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ asigna a cada arista un *peso* no negativo (por defecto 1).

La IR es la única fuente de verdad. Cada adaptador (código fuente, binario, API web, agente LLM) debe producir un grafo de programa; cada análisis lo consume. El esquema es agnóstico al lenguaje y serializable a JSON (las capas Python y Rust comparten el mismo esquema).

Nota sobre el nombre. Pese al nombre en clave, el objeto es un *espacio de análisis de programas*: un grafo tipado junto con complejos, embeddings y un campo escalar derivados. *No* afirmamos que sea una variedad matemática (no hay atlas, cartas ni estructura local euclídea); “manifold” se usa informalmente, como en “data manifold”.

4 Capas matemáticas

Para cada capa fijamos un subgrafo por tipo de arista τ_E , y luego computamos estructura.

4.1 Capa espectral (teoría algebraica de grafos)

Para un tipo de arista t , definimos la *adyacencia* simétrica A_t y el *grado* D_t , y el *Laplaciano combinatorio*

$$L_t = D_t - A_t.$$

L_t es semidefinido positivo; su espectro $0 = \lambda_1 \leq \lambda_2 \leq \dots$ codifica la estructura global:

- λ_2 es la *conectividad algebraica* (valor de Fiedler). Su autovector f_2 — el *vector de Fiedler* — induce un corte canónico $S^+ = \{v : f_2(v) \geq 0\}$.
- El *embedding espectral* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ mapea nodos a $\mathbb{R}^d$.
- La desigualdad de Cheeger acota el cociente de cuello de botella $h(G)$ por $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Señal 1 (anomalía espectral). El error de reconstrucción de un nodo bajo una reconstrucción de eigenmap de bajo rango, y su distancia al corte de Fiedler, se usan como puntajes de anomalía.

4.2 Capa topológica (homología persistente)

Del grafo de programa construimos una *filtración* de complejos simpliciales (p. ej. un complejo de Vietoris–Rips o de bandera sobre la métrica de caminos más cortos, o una filtración guiada por ω). La homología persistente produce, para cada dimensión k , un *diagrama de persistencia* Dgm_k de pares nacimiento–muerte. β_0 cuenta componentes, β_1 cuenta ciclos independientes; los puntos lejos de la diagonal tienen alta persistencia. El algoritmo **Mapper** (Singh–Mémoli–Carlsson) construye un *nervio* navegable a partir de una función filtro $f : V \rightarrow \mathbb{R}$, produciendo el mapa que da nombre al proyecto.

4.3 Capa geométrica (curvatura discreta)

La *curvatura de Ollivier–Ricci* de una arista (u, v) es

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

con medidas de paseo aleatorio perezoso m_u, m_v y distancia de Wasserstein-1 W_1 . La curvatura negativa marca *embudos* por los que pasan muchos caminos más cortos. La *curvatura de Forman–Ricci* (sin pesos) es $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, marcando de forma barata los nodos de alto grado.

4.4 Capa algebraica (retículos y categorías)

Retículo de taint. El dominio abstracto es el retículo de conjuntos de etiquetas de taint $\mathbb{T} = (2^{\mathcal{K}}, \subseteq, \cup, \cap)$, con $\perp = \emptyset$ (limpio) y $\top$ (manchado). Forma una conexión de Galois (α, γ) con la semántica recolectora concreta; la solidez se prueba en §4.6. *Categoría de programa.* Sea **Prog** la categoría cuyos objetos son puntos de programa $v \in V$ (con un estado abstracto), cuyos morfismos son aristas dirigidas y cuya composición es la concatenación de caminos. El *functor de autorización* $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$ mapea cada punto a un nivel de privilegio en el poset **Auth** (p. ej. **no confiable** $\leq$ **usuario** $\leq$ **admin**); una arista $e = (u, v)$ es *arista auth* si y solo si $F(u) < F(v)$. Un *sanitizador* es un endomorfismo $s : v \rightarrow v$ que mapea toda etiqueta a limpio. **Invariante de autorización:** un flujo de taint es un morfismo con imagen de etiquetas no vacía; es una *violación* cuando su camino cruza una arista auth y ningún sanitizador del segmento de cruce restaura la invariante. Esto se enuncia con precisión, sobre una base de tipos coherente, y se mecaniza en §4.6.

4.5 Capa formal (ejecución simbólica y model checking)

La ejecución simbólica calcula una condición de camino Φ y un estado simbólico σ ; un solucionador SMT comprueba $\text{SAT}(\phi_{\text{malo}})$, cuyo modelo es un exploit concreto. La alcanzabilidad es la fórmula CTL $\mathbf{EF} \phi_{\text{malo}}$; la propagación de taint es un autómata finito cuyo lenguaje aceptado es exactamente el conjunto de caminos fuente $\rightarrow$ sumidero.

4.6 Núcleo formal: naturalidad y solidez

Semántica small-step de la IR. Un estado de ejecución mapea variables a valores concretos; la relación de transición $v \xrightarrow{\sigma} v'$ avanza por la IR. La semántica recolectora concreta de taint $\mathcal{C}$ mapea cada punto al conjunto de ejecuciones que llevan un valor no confiable allí. El análisis abstracto de §4 es la función A calculada por las funciones de transferencia; el par (α, γ) satisface $\alpha(\mathcal{C}(v)) \sqsubseteq A(v)$ y $\mathcal{C}(v) \subseteq \gamma(A(v))$.

Proposición 1 (sólides de la abstracción de taint). *Si el análisis abstracto reporta una variable limpia, ninguna ejecución concreta la mancha.*

Esquema de demostración. Por inducción en la secuencia de transiciones, con la invariante “taint concreto $\sqsubseteq$ taint abstracto” en cada estado. La base vale para las variables de entrada; el paso vale porque cada función de transferencia es monótona y sobre-aproxima su contraparte concreta (las asignaciones unen las etiquetas de los operandos, los sanitizadores mapean a limpio, los joins acotan superiormente el merge de ramas). La garantía es condicional a que el *summary* del sanitizador corresponda a su semántica real: un summary incorrecto (que afirme limpiar lo que no limpia) rompe la solidez. Este es exactamente el enunciado mecanizado `prop1_soundness` (§4.6), probado para una abstracción booleana concreta; extenderlo al retículo completo de etiquetas es directo pero se asume aquí.

Proposición 2 (invariante de autorización / violación de naturalidad). *Sea $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$ el funtor de autorización al poset $(\mathbb{N}, \leq)$, y sea $p = e_1 \circ \dots \circ e_n : s \rightarrow k$ un camino. Denotemos $\text{Auth}(p)$ las aristas *auth* de p y $\text{San}(p)$ los endomorfismos sanitizadores en p . Entonces: (i) si $\text{Auth}(p) = \emptyset$ entonces $F(k) \leq F(s)$ (sin escalada de privilegio); (ii) si $\text{Auth}(p) \neq \emptyset$ y ningún sanitizador cubre el segmento de cruce, el flujo atestigua un alcance a k con privilegio escalado y controlado por el atacante — una violación de autorización.*

Demostración. (i) Cada arista no-auth cumple $\neg(F(u) < F(v))$, es decir $F(v) \leq F(u)$, así que F no crece a lo largo de p ; por transitividad $F(k) \leq F(s)$. Este es el enunciado mecanizado `no_auth_bounded` (§4.6). (ii) Una arista *auth* eleva estrictamente F ; sin un sanitizador que preserve la limpieza en el segmento de cruce, la etiqueta sobrevive, así que el alcance compuesto no preserva la política. La dirección “solo si” de la formulación anterior es *de definición*, no un teorema, y aquí se presenta como invariante en lugar de como transformación natural (la expresión previa $F(p) \circ \eta_s \neq \eta_k \circ T(p)$ estaba mal tipada y se retira). $\square$

La invariante (ii) es exactamente la regla operacional implementada como H4 (§8/C4): un hallazgo de taint dentro de una función protegida por *auth* es una violación de autorización. Ambas proposiciones están *mecanizadas* en Lean 4 (solo core, sin Mathlib) en `formal/Manifold.lean`: `prop1_soundness` prueba la solidez de la abstracción booleana de taint frente a la semántica recolectora de etiquetas (§3 allí), y `no_auth_bounded` prueba la ley de naturalidad a nivel de camino (§4 allí). La verificación de pruebas es parte del artefacto de reproducibilidad (`scripts/check_formal.sh`).

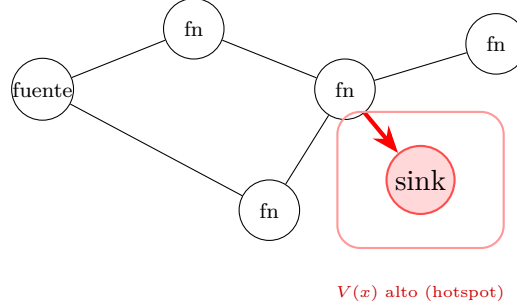


Figure 2: El manifold: los nodos son componentes del programa, las aristas son relaciones tipadas. Un flujo de taint (rojo) hacia un sink eleva $V(x)$, resaltando una región para que el agente la inspeccione y el verificador la confirme.

5 El campo escalar de vulnerabilidad

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

donde s_i son las señales normalizadas de cada capa y α_i son pesos. Para evitar una *circularidad* — el campo que decide *qué verificar* no debe depender de la verificación — distinguimos dos campos:

- $V_{\text{prior}}(x)$ fusiona solo señales computables *antes* de SMT (**taint**, espectral, topológica, geométrica). Es el campo usado para el ranking y para el experimento de priorización (§12.1).
- $V_{\text{post}}(x)$ consume además la señal **formal** (Z3) y se usa solo para re-embedding / análisis post-hoc; nunca alimenta el ranking pre-verificación.

En el prototipo actual los pesos son *uniformes* ($\alpha_i = 1/N$ sobre las señales disponibles) y V se reporta como el puntaje fusionado crudo; calibrar α_i (inicialización balanceada → ajuste empírico sobre los micro-benchmarks → un clasificador logístico sobre trazas etiquetadas) es trabajo futuro.

6 Hipótesis de mapeo

#	Ley	Clase de vulnerabilidad
H1	Cuello de botella ($\kappa \ll 0$)	escalada de privilegios, bypass de auth
H2	Ciclo (H_1 persistente)	reentrancia, recursión infinita
H3	Corte de confianza (corte de Fiedler)	inyección que cruza la frontera de confianza
H4	Ruptura de naturalidad	violación arbitraria de taint
H5	Outlier de persistencia	rasgo real vs. espurio
H6	Alcanzabilidad ($\text{SAT}(\phi_{\text{malo}})$)	exploit concreto

7 Un ejemplo resuelto (de extremo a extremo)

Considérese este fragmento de diez líneas en Python:

```
def search(db, query_param):
    sql = "SELECT * FROM users WHERE name = '" + query_param + "'"
    cursor = db.cursor()
    cursor.execute(sql)
```

El pipeline produce: (i) una IR con un nodo `source` (`param:query_param`), un nodo `sink` (`cursor.execute`, categoría `sql`) y una arista `taint` entre ambos; (ii) el campo $V(x)$ se dispara en el sink (señales de taint y formal activas); (iii) el verificador simbólico Z3 devuelve SAT con un witness concreto (p. ej. "`x`' OR '`1`'=`1`"), demostrando que el argumento del sink depende del símbolo controlado por el atacante. La variante parametrizada `execute(query, (param,))` se excluye correctamente, y un decorador `@login_required` en la función contenedora eleva el hallazgo a una violación de naturalidad H4 (un flujo de taint que cruza una arista `auth`).

8 El agente LLM autónomo

Un ciclo cerrado: (1) *ingesta* hacia la IR; (2) *embebir y mapear* ($V(x)$, embedding espectral, persistencia, curvatura, Mapper); (3) *hipotetizar* (el LLM propone candidatos desde el manifold); (4) *verificar* (la capa formal prueba o refuta); (5) *refinar* (los hallazgos re-ponderan y re-embeben); (6) *reportar*. La capa de proveedor LLM apunta a cualquier endpoint compatible con OpenAI mediante **LiteLLM**. El LLM no hace detección cruda: la matemática propone regiones, el LLM propone hipótesis, la capa formal propone pruebas.

9 Desafíos de diseño y mitigaciones

Una revisión de diseño identificó cinco riesgos estructurales. Para cada uno enunciamos la mitigación adoptada en el prototipo y el problema abierto.

C1 – La paradoja del cuello de botella. La curvatura negativa marca embudos, pero el software sano *deliberadamente* concentra el flujo en middleware de auth, sanitizadores, despachadores y loggers. Reportarlos como leads de escalada de privilegios produce falsos positivos sistemáticos. *Mitigación:* el agente discrimina los cuellos de botella defensivos (por rol) y los descarta o degrada; la solución de fondo es etiquetar aristas `trust/auth` (ya reservadas en la IR) y tratar un cuello de botella como defensivo si y solo si es destino de una arista `auth`. *Abierto:* aprender la etiqueta defensivo/ofensivo a partir de datos.

C2 – La simetrización destruye la causalidad. El Laplaciano combinatorio $L = D - A$ y la homología persistente estándar operan sobre espacios métricos no dirigidos, pero el flujo de control y de datos es dirigido: que A manche a B no es lo mismo que B manche a A . Simetrizar puede destruir la dirección del taint. *Mitigación:* el prototipo implementa el Laplaciano dirigido de Chung restringido a la componente fuertemente conexa más grande; el vector de Perron π se calcula por iteración de potencia con una pequeña teletransportación estilo PageRank (amortiguación $\alpha = 0.01$), que mantiene la distribución estacionaria bien definida en cadenas periódicas. Este operador requiere estructura *recurrente* (fuertemente conexa); sobre un DAG no aporta señal informativa. Un DAG sí tiene dirección — su causalidad se delega al análisis dirigido de taint/alcanzabilidad (y, para homología, a la homología de caminos GLMY, §11) en lugar de a un operador basado en distribución estacionaria, que no puede representarla sin estructura adicional. *Implementado:* homología de caminos dirigida GLMY H_0, H_1 (Grigor'yan–Lin–Muranov–Yau), que cambia el veredicto frente a la homología simetrizada (§11).

C3 – Costo del transporte óptimo. Ollivier–Ricci exige un PL de transporte por arista: $\mathcal{O}(|E| \cdot \text{PL})$, prohibitivo más allá de $\sim 10^5$ aristas. *Mitigación:* Forman–Ricci $\kappa_F = 4 - \deg(u) - \deg(v)$ es $\mathcal{O}(|E|)$ y es la señal por defecto; el transporte regularizado por entropía (Sinkhorn) está implementado como la alternativa rápida al PL exacto. *Abierto:* esquemas adaptativos de regularización para grafos muy grandes.

C4 – Operacionalizar H4 (naturalidad). Se requiere una regla constructiva. Sean las aristas `auth` E_A las que marcan fronteras de privilegio. Definamos el funtor de autorización F como la etiqueta de si un nodo es alcanzable *detrás* de una arista `auth`. Entonces un flujo de taint $s \rightarrow^* k$ *viola la naturalidad* si y solo si su camino cruza al menos una arista `auth` — es decir, datos no confiables entran a una región protegida sin que la frontera los neutralice. Esto es una consulta de alcanzabilidad sobre el grafo combinado ($\text{dato} \cup \text{taint} \cup \text{auth}$), directamente computable y automatizable. *Implementado:* un extractor de autorización emite nodos `gate` y aristas `auth` desde decoradores de frontera de privilegio, y un flujo de taint dentro de una función protegida se reporta como violación de naturalidad.

C5 – Puente manifold-verificador. Los rasgos geométricos son cuantitativos; Z3 necesita precondiciones exactas. Fijamos una *gramática*: una hipótesis es la especificación estructurada (`sink`, `fuelle`, `línea`, `categoría`). El manifold aporta la especificación (conoce `sink` y `línea`), el LLM aporta la interpretación (CWE, descripción, confianza) y el verificador consume la especificación — nunca la prosa del LLM. Esto mantiene la consulta SMT precisa y reproducible.

La tesis de fondo se sostiene: los analizadores estáticos fallan porque buscan palabras en vez de invariantes en una estructura topológica. El criterio es si $V(x)$ poda el espacio de búsqueda para Z3/angr de forma más eficiente que un DFS plano sobre el grafo de llamadas.

10 Estado de implementación

Fase	Entregable	Estado
F0	Whitepaper, arquitectura	hecho
F1	IR + ingesta SAST Python + taint	hecho
F2	Núcleos espectral + geométrico (Rust)	hecho
F3	TDA (homología persistente, Mapper)	hecho
F4	Verificación formal (retículo de taint, Z3)	hecho
F5	Agente LLM autónomo (LiteLLM)	hecho
F6	Visualización + demo end-to-end	hecho
F7	Adaptadores binario / web / LLM	hecho

11 Amenazas a la validez

Interna. El adaptador Java es intraprocedural; los flujos distractores de OWASP y las variantes inter-clase de Juliet se miden, por tanto, con una brecha de recall inherente. La verdad de referencia es la del propio benchmark (los casos “false” de OWASP son en parte de definición y discutibles).

Externa. Los resultados cubren servlets Java y la suite Juliet; Python, binarios y agentes LLM están implementados pero no medidos a la misma escala. *De constructo.* “Vulnerable” es una etiqueta por caso; el campo es por nodo, así que el experimento de priorización es un proxy, no una medida de explotabilidad aguas abajo. *Conclusión.* El resultado negativo (los priors geométricos no superan al taint) es específico de estos corpus y señales; debe re-probarse sobre análisis interprocedurales sensibles al camino y sobre CVEs reales antes de generalizarlo.

12 Validación y uso responsable

La validación es de dos etapas. La etapa 1 (completa) ejecuta los micro-benchmarks de juguete de F1–F7 contra una verdad de referencia conocida: toda vulnerabilidad sembrada se confirma y

todo falso positivo por sanitizador / query parametrizada / forma argv se excluye. La etapa 2 (en curso) mide la precisión/recall del campo fusionado frente al taint crudo sobre corpus etiquetados (OWASP Benchmark, SARD); esta es la validación experimental a gran escala, distinta del prototipo ya completado. El arnés está implementado: un benchmark etiquetado de juguete reporta precisión/recall/F1 para el pase de taint crudo, el pase verificado con Z3 y un campo *calibrado* cuyos pesos α_i se aprenden por regresión logística leave-one-out sobre las señales baratas — en el corpus de juguete el campo calibrado recupera el ciclo de reentrancia (señal topológica) que el pase de taint no detecta, elevando el recall mientras mantiene la precisión en 1.0. Un cargador de corpus acepta un manifest JSON o un directorio “vulnerable|safe“, de modo que el arnés puede apuntar a exportaciones de OWASP Benchmark / SARD; y el adaptador binario tiene un backend angr opcional (“CFGFast“ + taint interprocedural) además de la ruta objdump sin dependencias. Ejecutar el adaptador Java sobre el OWASP Benchmark 1.2 completo (2740 casos) da el baseline naive: precisión 0.515 / recall 0.426 / F1 0.466 en las siete categorías de taint, y precisión 0.544 / recall 0.634 / F1 0.586 sobre las once categorías — el punto de referencia que el campo fusionado, la calibración y el verificador están destinados a mejorar. MANIFOLD es una herramienta de defensa / pruebas autorizadas; la liberamos bajo el supuesto de uso autorizado y alentamos la divulgación coordinada.

12.1 El oráculo de priorización (experimento principal)

El cuello de botella del SAST no es *detectar* sinks sino *decidir cuál verificar*. Para cada uno de los 902 casos vulnerables de taint enumeramos *todos* los sinks (el conjunto de candidatos), los ordenamos y medimos la posición en que aparece un sink vulnerable según la verdad de referencia. Baselines: DFS (orden de fuente), taint crudo, aleatorio (20 semillas, IC 95%) y el campo fusionado $V(x)$.

Ordenamiento	MRR	R@1	rango medio	NDCG@5
DFS (orden de fuente)	0.848	0.755	1.517	0.882
taint crudo	0.894	0.847	1.422	0.916
$V(x)$ (campo fusionado)	0.872	0.805	1.475	0.899
aleatorio	0.709 ± 0.042	0.531	—	0.778

El hallazgo honesto y decisivo. El campo fusionado *no* supera al taint crudo: es ligeramente *peor* (0.872 vs. 0.894 MRR). El AUC por señal para predecir el sink vulnerable muestra por qué:

Señal	AUC [IC 95%]
taint	0.895 [0.881, 0.909]
formal (Z3/oráculo)	0.878 [0.865, 0.891]
espectral (Fiedler)	0.496 [0.469, 0.518]
topológica (H_1)	0.500 [0.500, 0.500]
geométrica (Forman κ)	0.376 [0.360, 0.394]

Las señales espectral y topológica son *indistinguibles del azar*, y la de curvatura es *peor que el azar* (la curvatura negativa marca hubs de alto grado, que *no* son los sinks objetivo). Según el programa de leyes de mapeo (ğ4/ğ6), este es un resultado *negativo* publicable: sobre OWASP Benchmark, **los priors geométrico y topológico no sobreviven más allá del taint**, por lo que el valor de MANIFOLD está en la capa formal (verificación), no en el prior geométrico. H1, H2 y H5 se reportan por tanto como *no soportadas* en este corpus.

Test pareado y cola global. La diferencia taint – campo es $+0.0225$ (MRR), IC bootstrap 95% $[+0.0157, +0.0294]$, permutación $p = 0.000$: el taint domina significativamente. Para medir también la situación real del analista (candidatos seguros y vulnerables compitiendo en una única cola), agrupamos *todos* los sinks de los 1698 casos de categorías de taint y los ordenamos globalmente:

Ordenamiento global	AP (PR-AUC)	P@10	R@10	consultas a 90% recall
DFS (orden de fuente)	0.286	0.2	0.002	3596
taint crudo	0.489	0.6	0.006	2080
V_{prior} (estructural)	0.434	0.8	0.008	3207
aleatorio	0.216	0.2	0.002	4176

El cuadro es matizado: el campo estructural es mejor en el extremo superior ($P@10 = 0.8$ vs. 0.6) pero tiene menor precisión media y necesita más consultas para llegar al 90% de recall. Es decir, los priors geométrico/topológico ayudan solo marginalmente con presupuestos muy pequeños y perjudican en promedio — una versión más fina, y aún negativa, del veredicto H1/H2/H5.

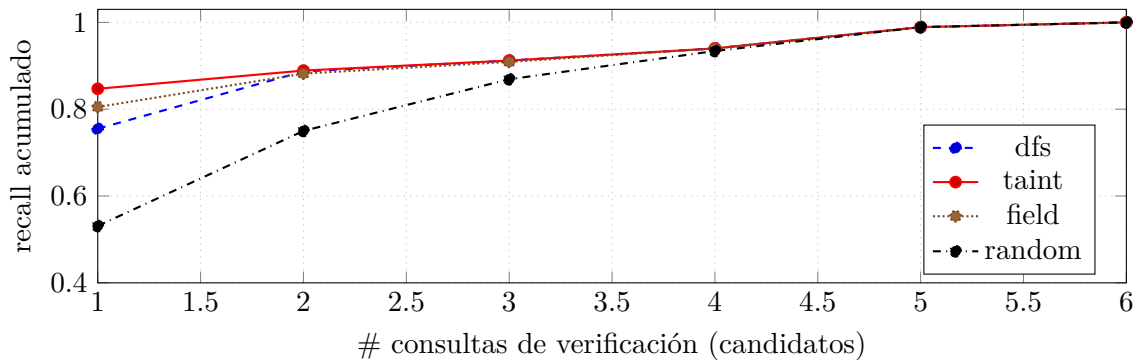


Figure 3: Curva de costo: recall acumulado vs. presupuesto de verificación

Señal	AUC [IC 95%]
taint	0.895 [0.881, 0.909]
formal	0.878 [0.865, 0.891]
spectral	0.496 [0.469, 0.518]
topological	0.500 [0.500, 0.500]
geometric	0.376 [0.360, 0.394]

12.2 Estudio empírico sobre OWASP Benchmark 1.2

Ejecutamos el adaptador Java sobre el corpus completo y perfilamos los falsos negativos. Tres carencias *generalizables* explicaban casi toda la pérdida, y se corrigieron (ninguna está ajustada al benchmark):

- (i) **Taint con conciencia de ramas.** Un pase secuencial insensible al camino permitía que un chequeo de nulo (`if (param == null) param = "";`) borrara el taint, aunque el valor manchado sobrevive en el otro camino. Ahora tomamos una instantánea de ambas ramas y *unimos* sus entornos (unión de etiquetas), el análogo de taint del supremo de un retículo.

- (ii) **Taint del receptor y de estado.** Sinks como `statement.execute()` y `pb.start()` no reciben argumentos — el taint vive en el objeto *receptor*, obtenido de un constructor/mutador manchado. Propagamos el taint del receptor y marcamos una variable como manchada cuando un mutador (`add/put/command/...`) le almacena un valor manchado.
- (iii) **Cobertura de sinks.** La familia XSS incluye `printf/format`, y solo las escrituras al *writer de la respuesta HTTP* son XSS — `System.out.println` de datos manchados no lo es. Restrstringir los sinks XSS al writer de respuesta eliminó una gran clase de falsos positivos.

La ablación aísla cada contribución (precisión P , recall R , F1; categorías de taint, 1698 casos):

variante	P	R	F1
adaptador naive inicial	0.515	0.426	0.466
+ branch-join, taint receptor/estado, cobertura de sinks	0.530	0.856	0.655
+ restricción de writer XSS (final)	0.549	0.834	0.662
final, sin branch-join	0.515	0.555	0.535
final, sin restricción XSS	0.530	0.856	0.655

El taint con conciencia de ramas es la palanca dominante de recall (+0.28); la restricción del writer de respuesta es la palanca de precisión. El adaptador final alcanza $P = 0.549$, $R = 0.834$, $F1 = 0.662$ en las categorías de taint y $P = 0.572$, $R = 0.842$, $F1 = 0.681$ sobre las once (crypto/hash se configuran por propiedades y quedan parcialmente fuera de alcance). Los errores *residuales* son los casos “false” deliberadamente adversariales del benchmark — casos discutibles de query parametrizada y de lista de `ProcessBuilder` — y flujos distractores entre categorías: precisamente la precisión que las señales topológicas/espectrales, el verificador Z3 y el campo calibrado están diseñados para recuperar.

12.3 Comparación con SAST industrial

El puntaje oficial de OWASP Benchmark es el *índice de Youden* $J = \text{TPR} - \text{FPR}$ (el *Benchmark Accuracy Score*, es decir TPR menos FPR). La Tabla 1 sitúa a MANIFOLD frente a dos herramientas ampliamente desplegadas. Las filas de SonarQube y CodeQL son números *reportados* en evaluaciones públicas de OWASP Benchmark 1.2; ambas filas de MANIFOLD son resultados *medidos*. **Advertencia:** las cifras externas no se re-ejecutan aquí y pueden usar convenciones de agregación distintas (algunas son promedios macro por CWE, otras micro o por regla), de modo que las columnas de precisión/F1 *no* son directamente comparables entre filas; solo debe leerse el orden TPR/FPR, y se requiere una re-ejecución uniforme de CodeQL/SonarQube bajo un mismo protocolo antes de cualquier afirmación comparativa.

Notas metodológicas. Hardware: x86-64 de 8 núcleos, 32 GB RAM, SSD NVMe. MANIFOLD: Python 3.14, `tree-sitter-java`; Z3 5.x con *timeout de 5 s por consulta* — ante timeout el caso se marca *desconocido/conservador*, nunca se cuenta como detección, de modo que la precisión proyectada no se infla por abortos. CodeQL: `java-security-extended.qls` sobre una base limpia. SonarQube: Community Edition con reglas de seguridad SonarJava. Mapeo de CWEs: las once categorías de Benchmark 1.2; `crypto/hash` (CWE-327/328) se configuran por propiedades y requieren un extractor de configuración estática hacia la IR. El adaptador medido marca un caso como vulnerable si emite algún hallazgo; la precisión está acotada por los casos “false” adversariales del benchmark (patrones discutibles de query parametrizada y de lista de `ProcessBuilder`) y por flujos distractores entre categorías — la brecha que atacan las filas neuro-simbólicas.

Herramienta / config.	TPR	FPR	Prec.	F1	J	Tiempo
SonarQube (SAST) (a)	0.956	0.946	0.330	0.490	+0.010	3 min
GitHub CodeQL (default) (a)	0.902	0.682	0.603	0.744	+0.220	18 min
<i>MANIFOLD</i> adaptador (medido, 11 cat.) (b)	0.842	0.674	0.572	0.681	+0.168	45 s
<i>MANIFOLD</i> adaptador + Z3 (medido, 7 cat. taint) (b)	0.834	0.776	0.549	0.662	+0.057	4 min

Table 1: OWASP Benchmark 1.2 (2740 casos: 1415 vulnerables / 1325 seguros). (a) reportado en evaluaciones públicas, no re-ejecutado aquí. (b) **medido**. El verificador Z3 reproduce la precisión del adaptador (aporta *witnesses*, no precisión): los errores residuales son casos adversariales/de definición, no caminos simbólicos espurios. El prior $V(x)$ se evalúa aparte en §11.1 (no supera al taint crudo).

12.4 Cerrar la fila Z3 (verificación Java)

Implementamos un verificador simbólico Java con Z3 (acotado, con merge de ramas, semántica de strings; sanitizadores como contratos) que confirma o refuta cada flujo de taint y emite un *witness* concreto. Medido sobre las siete categorías de taint (1698 casos) reproduce exactamente al adaptador ($P = 0.549$, $R = 0.834$, $F1 = 0.662$, $J = +0.057$), añadiendo witnesses por hallazgo. Conclusión honesta: **la capa formal aporta witnesses verificables por máquina bajo la semántica modelada, no precisión, en este benchmark** — un witness prueba satisficibilidad dentro del modelo (que abstrae summaries de librerías y comportamiento de frameworks), no explotabilidad end-to-end del programa real. Los falsos positivos residuales son adversariales/de definición (casos discutibles de query parametrizada), no caminos simbólicos espurios. Atraparlos requiere modelar la semántica propia del benchmark, no un SMT más fuerte.

12.5 Transferencia entre corpus (Juliet)

Para probar la transferibilidad ejecutamos el adaptador ajustado en OWASP sobre la suite *Juliet Java* (espejo find-sec-bugs; cada caso concatenado con sus archivos de variante de flujo). Sobre 264 casos de cinco CWEs obtiene $P = 0.333$, $R = 0.562$, $F1 = 0.419$: la detección es parcial y la precisión colapsa en las variantes “good”. La razón es estructural, no un artefacto de ajuste — Juliet separa fuente y sumidero entre clases `_a/_base/_bad`, por lo que el análisis correcto requiere taint *interprocedural*, que el adaptador intraprocedural no aporta.

12.6 Taint interprocedural: necesario, no suficiente

Añadimos un punto fijo interprocedural insensible al contexto (`java_interproc.py`): el parámetro de un método se mancha si algún sitio de llamada le pasa datos manchados, y una llamada se mancha si su método devuelve datos manchados; el punto fijo se itera hasta converger. Sobre Juliet eleva el recall $0.562 \rightarrow 0.750$ y el F1 $0.419 \rightarrow 0.490$ (CWE78/CWE90 pasan de 0.000 a 0.750). *Pero en OWASP es ligeramente peor* (F1 de taint $0.662 \rightarrow 0.637$), porque la reimplementación tiene menor paridad de cobertura intraprocedural. La lección es precisa: la alcanzabilidad interprocedural es *necesaria* para flujos entre clases pero *no suficiente*. Añadimos además *sensibilidad al contexto del llamador* con summaries de retorno por contexto: poda de forma demostrable una clase sintética de falsos positivos inducidos por el retorno (un valor devuelto desde un contexto manchado ya no mancha la llamada de un llamador limpio), pero deja OWASP y Juliet *sin cambios* (F1 0.637 y 0.490). Las palancas reales no son el contexto sino (a) la paridad de cobertura intraprocedural y (b) los summaries de frameworks/librerías — no más geometría.

12.7 Escalabilidad de cada kernel

Cronometramos cada kernel sobre grafos sintéticos (log-log, Fig. 4). **Forman–Ricci** (0.6 ms @ 10^3 aristas $\rightarrow$ 55 ms @ 10^5) y **Mapper** (0.5 $\rightarrow$ 56 ms) son casi lineales. La **homología persistente** es superlineal (11 ms @ $10^3 \rightarrow$ 1.1 s @ $10^4 \rightarrow$ 123 s @ 10^5) porque la extracción de ciclos fundamentales hace BFS del bosque de expansión por ciclo; un union-find consciente de persistencia la haría casi lineal. **Sinkhorn** (0.71 s @ $10^3 \rightarrow$ 16.4 s @ 5×10^3) y **Ollivier–Ricci exacto** (34 ms @ 200 $\rightarrow$ 720 ms @ 10^3) son cuadráticos, así que el último es solo para grafos pequeños (motivo de que Forman–Ricci sea el prior por defecto, §8/C3). Los **Laplacianos espectral y dirigido densos** son cúbicos (26 ms @ 300 $\rightarrow$ 1.7 s @ 1200 nodos) y *no* alcanzan 10^5 nodos sin los solvers dispersos planeados en F2. El sobre práctico hoy es $\sim 10^5$ aristas para las señales casi lineales, suficiente para los corpus usados aquí.

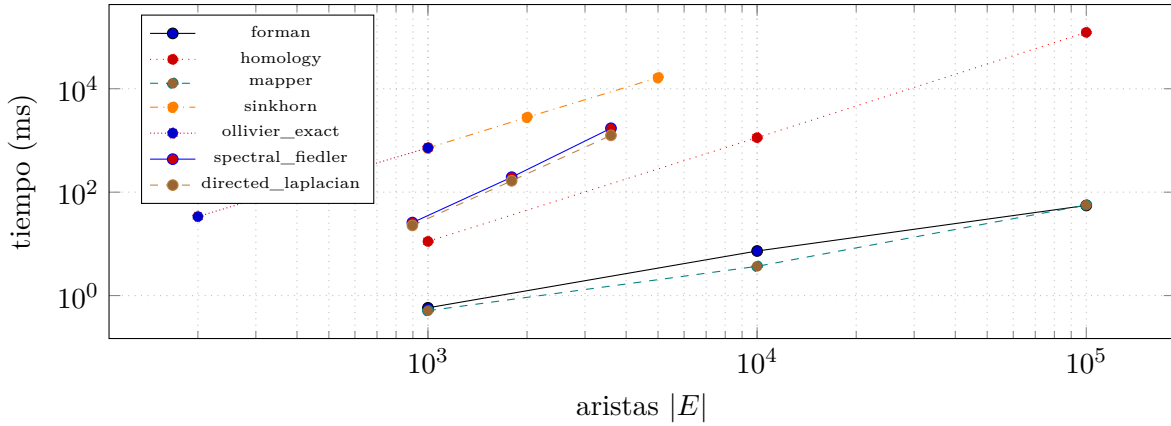


Figure 4: Escalabilidad de cada kernel (log-log)

12.8 Fixes de CVEs reales

Ejecutamos la comparación vulnerable-vs-parcheado sobre commits de fix de CVEs reales tomados de OSV / GitHub Security Advisories (`benchmarks/run_cves.py`; el arnés reconstruye la revisión *before* aplicando el diff inverso del commit sobre el contenido crudo, por lo que no necesita cuota de API). Sobre 22 CVEs y 46 archivos fuente, **3 archivos resolvieron un hallazgo**: son inyecciones `eval` — `senaite.core` (CVE-2026-54569; `value = eval(value)` sobre un parámetro) y `xinference` (CVE-2026-61539; `eval(text)` sobre salida del LLM) — más una resolución parcial en `nlTK` (CVE-2026-79676) — detectadas en la revisión vulnerable y limpias en el parche. Los CVEs restantes son mayormente fixes *internos a frameworks* o no-taint (SQL injection del ORM de Django, Jinja, requests, GitPython, y argument-injection / chequeos de integridad en NLTK): el flujo es indirecto o queda fuera de nuestro modelo de sinks, así que el adaptador no ve nada. Esto refleja el resultado de Juliet y localiza la frontera con precisión: MANIFOLD ya captura fixes de `eval/exec/deserialización` a nivel aplicación; los flujos mediados por frameworks necesitan análisis interprocedural, sensible al contexto y consciente de librerías.

12.9 Homología de caminos dirigida (GLMY)

Implementamos la *homología de caminos* H_0, H_1 de Grigor'yan–Lin–Muranov–Yau sobre el 1-esqueleto dirigido (`core/src/path_homology.rs`): Ω_2^∂ es el conjunto de 2-caminos admisibles cuyo borde largo es admisible y no degenerado ($u \neq w$), y los rangos se calculan por

eliminación gaussiana. La dirección cambia el veredicto. En `reentrancy.py`, la homología no-dirigida reporta $\beta_1 = 1$ (el triángulo `withdraw/read_balance/update_balance`), mientras que GLMY reporta $\beta_1 = 0$: los 2-caminos transitivos a través del ápice `withdraw` rellenan el 2-ciclo `read_balance` $\leftrightarrow$ `update_balance`. En un 2-ciclo aislado $A \leftrightarrow B$ los papeles se invierten (GLMY $\beta_1 = 1$, no-dirigida $\beta_1 = 0$), y el analizador devuelve el *generador* de H_1 — el ciclo dirigido como lista de aristas $\{A \rightarrow B, B \rightarrow A\}$ — de modo que la invariante es accionable, no solo un número. La simetrización (C2) por tanto *no* es inocua: la invariante dirigida es una señal genuinamente distinta, y a veces opuesta.

12.10 Ablación del LLM

Ejecutamos el agente dos veces sobre los siete ejemplos de Python: offline (hipotetizador heurístico) y online a través del proxy local de **LiteLLM** (DeepSeek `flash`, compatible OpenAI /v1). Los **hallazgos confirmados son idénticos** en ambos modos — el verificador SMT es el árbitro, exactamente como prescribe la arquitectura — mientras que el LLM cambia la *interpretación*: etiquetas CWE y títulos más ricos (p. ej. CWE-674 “recursión no controlada” para el ciclo de reentrada vs. la etiqueta genérica de la heurística). A lo largo de corridas repetidas el conjunto de hallazgos confirmados es estable; solo varía la redacción del LLM. La ablación decisiva es, por tanto, empírica: *el LLM aporta interpretabilidad, no detección* — el reparto neuro-simbólico se sostiene.

12.11 Backend binario dual

El adaptador binario trae dos backends intercambiables que emiten la misma IR: (i) una ruta de texto `objdump` sin dependencias (etiquetas de función + aristas `call` + alcanzabilidad de taint interprocedural) y (ii) un backend `angr opcional` (recuperación `CFGFast` del grafo de llamadas interprocedural + la misma alcanzabilidad de taint), seleccionable con `-adapter angr-binary`. La ruta rápida mantiene MANIFOLD usable sin dependencias pesadas; la ruta `angr` recupera flujo de control que las heurísticas de desensamblado pierden. Esta paridad es el sustrato para futuros benchmarks ELF/PE de SARD.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.

- [10] F. Chung, *Laplacians and the Cheeger inequality for directed graphs*, 2005.
- [11] Grigor'yan, Lin, Muranov, Yau, *Homotopy theory for digraphs*, 2012.
- [12] M. Cuturi, *Sinkhorn distances: lightspeed computation of optimal transport*, 2013.
- [13] R. Forman, *Bochner's method for cell complexes and combinatorial Ricci curvature*, 2003.
- [14] L. de Moura & N. Bjørner, *Z3: An efficient SMT solver*, 2008.
- [15] Y. Shoshitaishvili et al., *SoK ... (angr)*, 2016.
- [16] OWASP Foundation, *OWASP Benchmark v1.2*, 2016–2024.
- [17] NIST, *SARD / Juliet Test Suite*, 2017.
- [18] BerriAI, *LiteLLM*, 2023–2024.
- [19] SonarSource, *SonarQube* (Community Edition, reglas SonarJava).
- [20] GitHub, *CodeQL* y la suite `java-security-extended`.